

Programozás alapjai **2** nagy házi feladat

- *Egyetemi útvonaltervező*



Készítette: Woynárovich Gergely

Specifikáció:

- A projekt célja:
Létrehozni egy útvonal tervező programot az egyetemi közlekedéshez. A program az egyetemet egy gráfként tárolva egy optimális útkereső algoritmussal a legrövidebb utat keresi egy A kezdőpontból egy B végpontba. Az optimális útvonalat utána kiírja, hogy végig mehessünk rajta.

- Funkciók:

- A gráf éleit és csúcsait egy külső txt-fileből olvassa be a program megnyitáskor. ; -vel elválasztva van megadva a pontok, és élek, hosszall együtt. Bemenet formátuma.

[PONTOK]

ID;Név

1;K épület főbejárat

2;K épület hátsóbejárat

3;Q épület bejárat

4;Könyvtár bejárat

[VONALAK] # ID;Név;Honnan_ID;Hova_ID;Hossz(m)

101;Főút;1;2;150

102;Parki sétány;2;3;300

103;Rövid átjáró;3;4;80

104;Hosszú körút;1;4;500

- Kereső algoritmus (pl. Dijkstra) kezeli a súlyozott éleket.
- Kiírja a legrövidebb utat csomópont és élenként. Ezentúl kiírja az út hosszát és a megtételéhez szükséges időt. A program kezdéskor felajánl 3 séta tempót, amiből az időt fogja számítani.
- Kiírja az összes csomópontot, hogy a felhasználó megnézhesse mikből választhat.
- Hozzálehet adni új csomópontokat a meglévő adatbázishoz. A program megnyitáskor megnyitja a txt-ből a térkép gráfját és eltárolja, amikor hozzáadunk új utakat azzal kibővíti ezt a listát és a program bezárásakor elmenti az új vagy megmaradt listát a txt-be.

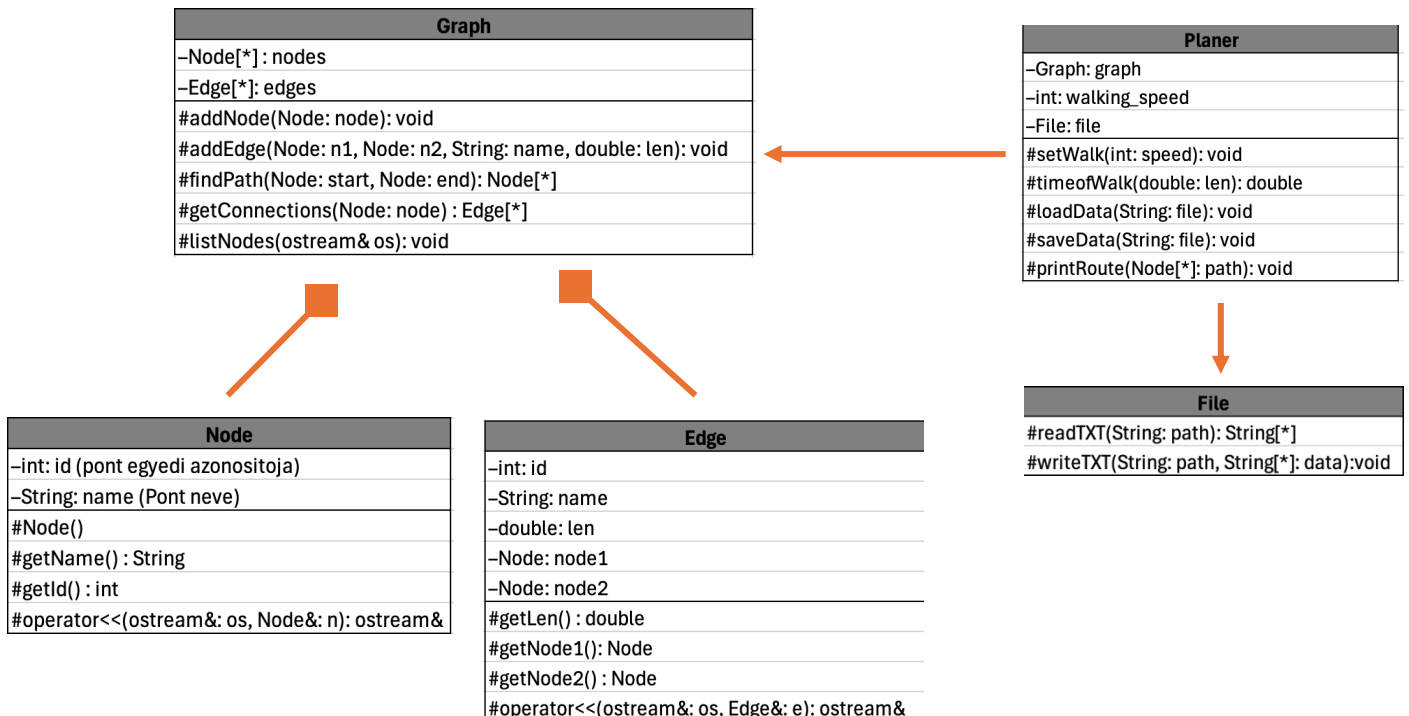
- Adatbekérés/Interakció:

- Minden adatot a billentyűzetről kér be. A felhasználó is a billentyűzeten tud parancsokat adni
(pl. Menüből alfunkció kiválasztása, 1.Útvonaltervezés, 2.Új pont felvétele, 3.Csomópontok listázása, 4.kilépés).
- A program az összes visszajelzést a terminálban jelzi vissza a felhasználónak.
(pl.: Leggyorsabb útvonal x-ből y-ba: zzz méter, zz perc, kezdőpont -> köztespont -> köztespont -> végpont.)

- Hibakezelés:

- Érvénytelen pontok megadásakor visszajelez, hogy ilyen nem létezik.
- Nem enged „szigetet” hozzáadni a gráfhoz.
- Ha nem találja a bemeneti file-t akkor kiírja, hogy nem talált és a program bezárásakor, létrehoz egy kimeneti file-t, amibe menti a potenciális újonnan megadott pontokat.
- Nem lehet azonos nevű pont vagy él.

Terv:



(# a privát típus, kivéve loadData, saveData, FindPath ami public)

- A program egy main programban kezdődik, ami létrehoz egy plannert és benne fut a menü egy while ciklusban. Innen tudja a felhasználó input megadásával belépni a program funkciójába.
- A program a Graph::findPath() függvényben egy Dijkstra algoritmussal keresi a legrövidebb utat a megadott két pont között.
 - Bemenet: két pont.
 - Minden ponthoz végtelen távolságot rendelünk kivéve a kezdőponthoz, ami 0 lesz.
 - Az összes többi pont meglátogatatlanak jelöli a program.
 - A kezdőpontból elérhető pontok közül kiválasztja a legközelebbit.
 - Megnézi az összes szomszédját milyen távol van és kiszámolja a távolságot odáig a kezdőpontból.
 - Ha ez a táv kisebb, mint az eddig abba a pontba menő táv akkor átírjuk.
 - Megjelöljük az előbb vizsgált pontot (amiből jött a következő) meglátogatottnak
 - Ezt addig ismétljük amíg el nem érjük a célpontot vagy el nem fogynak a meglátogatható pontok
 - Kimenet: Tömb a pontokból, amin végig kell menni.